Dyer
Innovation

CLAUDE CERTIFIED ARCHITECT · SCENARIO 4

Multi-Agent Research System

A study guide for hub-and-spoke orchestration, subagent context isolation, MCP structured errors, and provenance through synthesis.

LAST UPDATED

April 27, 2026

Contents

01 PART I – Core Architectural Patterns 3

1.1 Architectural Patterns 3

02 PART II – Deep Domain Coverage 4

2.1 The Agentic Loop Lifecycle (Task 1.1) 4
2.2 Subagent Spawning and Context Passing (Task 1.3) 4
2.3 Agent SDK Hooks (Task 1.5) 4
2.4 Programmatic Prerequisites and Handoffs (Task 1.4) 4
2.5 Task Decomposition Strategies (Task 1.6) 4
2.6 Session State, Resumption, and Forking (Task 1.7) 4
2.7 MCP Structured Error Responses (Task 2.2) 4
2.8 tool_choice Configuration (Task 2.3) 4
2.9 MCP Server Configuration (Task 2.4) 4
2.10 Verbose Tool Output Management (Task 5.1) 4
2.11 Context Management in Extended Sessions (Task 5.4) 4
2.12 Human Review Workflows and Confidence (Task 5.5) 4
2.13 Information Provenance (Task 5.6) 4

03 PART III – Quick Reference and Anti-Patterns 9

3.1 Anti-Patterns to Recognize on Sight 9
3.2 Core Terminology 9
3.3 15 Guiding Principles 9

01 PART I – Core Architectural Patterns

2. Architectural Patterns

These patterns represent the core mental models the CCA expects across the Multi-Agent Research scenario. The focus throughout is on why distractor answers are tempting and where they break down.

2.1 Hub-and-Spoke (Mediator) Pattern

Definition: A central coordinator manages all communication between specialized subagents. No subagent communicates directly with another.

Primary value:

- Centralized observability
- Consistent error handling
- Controlled information flow

> **TIP**
>
> Exam tip:
>
> When a question asks about the "main advantage" of routing through a coordinator, the answer is always about observability and governance — not performance or serialization.

2.2 Subsidiarity in Error Handling

Definition: Handle errors at the lowest level capable of resolving them. Subagents recover from transient failures locally; permanent failures escalate with full context.

Key judgment — distinguish:

- **Transient failures** (e.g., timeout) → retry locally
- **Permanent failures** (e.g., corrupted PDF) → escalate with context

Rules:

- Never retry permanent failures
- Never escalate routine transient failures

2.3 Structured Error Context

Errors between agents should include:

- Failure type
- What was attempted
- Any partial results
- Potential alternative approaches

> **TRAP**
>
> **Anti-pattern:**
>
> Generic statuses like "search unavailable" hide valuable context from the coordinator.

2.4 Semantic Error Classification

Distinguish:

- **Access failures** (e.g., timeout — answer unknown, may need retry)
- **Valid empty results** (query succeeded, found nothing — not an error)

These require fundamentally different coordinator responses.

2.5 Principle of Least Privilege for Agent Tools

- Give agents only the tools needed for their role
- Constrain behavior through tool capabilities (what the tool can do) rather than prompt instructions (what the agent is told to do)
- Structural guardrails beat behavioral guidelines

This is the single most important mental model for the exam: code guarantees; prompts suggest.

2.6 Tool Description Disambiguation

- Tool names and descriptions are the primary mechanism driving LLM tool selection
- When descriptions overlap (e.g., "analyzes content" vs "analyzes documents"), rename and redescribe both tools to create clear semantic boundaries
- Fix at the definition level, not through compensating classifiers or few-shot examples

2.7 Context Management Patterns

Lost-in-the-middle:

- LLMs attend more strongly to content at the beginning and end of long inputs
- Mitigate with frontloaded key findings summaries and explicit section headers
- Don't just truncate or reorder

Fix at the source:

- When combined outputs exceed downstream budgets, redesign upstream agents to return structured, concise data (key facts, citations, relevance scores) instead of verbose content
- Shift left

Graceful degradation:

- Proceed with available data while annotating coverage gaps
- Produce value with honest caveats rather than blocking on completeness or hiding incompleteness

Coordinator planning:

- When all agents succeed but output is incomplete, the root cause is the coordinator's task decomposition
- Partition research space upfront to prevent overlap

---

02 PART II – Deep Domain Coverage

3. The Agentic Loop Lifecycle (Task 1.1)

This is the most fundamental concept in the Agent SDK and the exam expects you to know it cold.

How It Works

The agentic loop is a while loop that sends messages to Claude, inspects the response's `stop_reason` field, and decides whether to continue or terminate.

When `stop_reason` is `"tool_use"`:

- Claude wants to call a tool
- Your code executes the requested tool
- Append the tool result to the conversation history
- Send the updated conversation back to Claude for the next iteration

When `stop_reason` is `"end_turn"`:

- Claude has decided it's finished
- The loop terminates and returns the final response to the user or coordinator

Example Flow

Coordinator sends: "Research the impact of AI on healthcare."

- **Turn 1:** Claude returns `stop_reason: "tool_use"` with a Task tool call to spawn the web search subagent. Code executes the Task tool and appends the subagent's findings as a `tool_result`.
- **Turn 2:** Claude sees the findings, returns `stop_reason: "tool_use"` with another Task call to spawn the document analysis subagent. Code executes and appends results.

- **Turn 3:** Claude returns `stop_reason: "tool_use"` to invoke the synthesis subagent with combined findings.
- **Turn 4:** Claude returns `stop_reason: "end_turn"` with the final synthesized report. Loop terminates.

Anti-Patterns to Know

Exam trap — distractors will suggest alternatives to `stop_reason`-based loop control:

- **Parsing natural language signals** (e.g., checking if the assistant says "I'm done") — fragile, non-deterministic
- **Arbitrary iteration caps** as the primary stopping mechanism (e.g., "stop after 5 iterations") — may cut off the agent before it finishes or waste iterations after it's done
- **Checking for assistant text content** as a completion indicator — Claude may include text alongside `tool_use` blocks, so presence of text doesn't mean it's finished

Correct approach:

- Continue on `"tool_use"`
- Terminate on `"end_turn"`
- Deterministic and model-driven

4. Subagent Spawning and Context Passing (Task 1.3)

The Task Tool

In the Claude Agent SDK, subagents are spawned using the **Task** tool.

- This is **not optional** — it is the mechanism for creating subagents
- For a coordinator agent to spawn subagents, its `allowedTools` configuration must include `"Task"`
- Without this, the coordinator cannot delegate

Isolated Context – Critical Concept

TRAP

TRAP

Exam trap:

— An answer that has the coordinator pass "a summary of findings" without structured metadata will lose source attribution

— The correct answer always preserves claim-source mappings through every handoff

Fork-Based Session Management

- The `fork_session` API creates independent branches from a shared analysis baseline
- In the research scenario, the coordinator might fork after initial data gathering to explore two different synthesis approaches in parallel, then compare results
- Each fork has its own conversation history from the fork point forward but shares the history before the fork

5. Agent SDK Hooks (Task 1.5)

Hooks are programmatic interceptors that run before or after tool execution. They provide deterministic guarantees that prompt instructions cannot.

PostToolUse Hooks – Data Normalization

A `PostToolUse` hook intercepts tool results before the model processes them. In the research system, different MCP tools may return data in different formats:

- One source returns dates as Unix timestamps
- Another as ISO 8601 strings
- A third as "March 2024"

A `PostToolUse` hook normalizes all dates to ISO 8601 before the agent sees them, ensuring consistent downstream processing.

EXAMPLE

Example application:

- The model picks up where it left off with the full conversation history

When to resume vs start fresh:

- **Resume** when prior context is mostly valid
- **Start fresh** with an injected summary when prior tool results are stale (e.g., the documents have been updated since the last session, or web search results may have changed)
- **Risk:** stale tool results in a resumed session can lead the model to make decisions based on outdated information

fork_session

- Creates independent branches from a shared analysis baseline
- After the coordinator gathers initial research, it might fork to explore two synthesis strategies:
  - One that prioritizes recency
  - Another that prioritizes source authority
- Each fork proceeds independently
- The coordinator can then compare the forked outputs and select the better approach

Exam application: If asked about exploring divergent approaches from a shared baseline without contaminating one exploration with the other → `fork_session` is the answer.

9. MCP Structured Error Responses (Task 2.2)

The isError Flag

- MCP tool responses include an `isError` boolean flag
- When a tool fails, set `isError: true` and include structured metadata in the response
- The agent uses this flag to determine that the tool call failed and that it needs to reason about recovery

Structured Error Metadata

A well-designed MCP error response includes three fields:

- `errorCategory` — one of:
  - **transient** (timeout, rate limit — may succeed on retry)
  - **validation** (bad input — fix the input)

TRAP

Exam trap:

— A generic "Operation failed" response without error categorization prevents the agent from making appropriate recovery decisions

— The correct answer always returns structured error metadata

10. tool_choice Configuration (Task 2.3)

The `tool_choice` parameter controls how Claude selects tools. This is a high-frequency exam topic.

SETTING	BEHAVIOR	WHEN TO USE
`"auto"`	Model may return text instead of calling a tool. It decides whether a tool call is needed.	Default for conversational agents. Use when the model should decide whether to use tools.
`"any"`	Model MUST call a tool but can choose which one. Cannot return text-only response.	Guaranteeing structured output when you have multiple extraction schemas and the doc type is unknown.
`{"type":"tool","name":"..."}`	Model MUST call the specified tool. No choice involved.	Forcing a specific extraction step first (e.g., `extract_metadata` before enrichment tools).

Tool Count and Selection Reliability

- Giving an agent access to too many tools (e.g., 18 instead of 4–5) degrades tool selection reliability by increasing decision complexity
- Each subagent should have only the tools relevant to its role
- If the synthesis agent has access to web search tools alongside its synthesis tools, it will drift into performing searches instead of synthesizing
- **Rule:** restrict tool sets per agent role

11. MCP Server Configuration (Task 2.4)

Scoping: Project vs User Level

Project-level:

- Configure in `.mcp.json` at the project root
- Shared with all team members via version control
- Use for production MCP servers that the entire team needs (e.g., the research system's web search server, document store server)

User-level:

- Configure in `~/.claude.json`
- Personal to one developer, not shared via version control
- Use for experimental MCP servers or personal tools (e.g., a personal notes server)

Environment Variable Expansion

- MCP server configurations in `.mcp.json` support environment variable expansion using `${VARIABLE_NAME}` syntax
- Allows credential management without committing secrets to version control
- Example: a search API key would be configured as `${SEARCH_API_KEY}` in `.mcp.json`, with each developer setting the actual key in their local environment

MCP Resources vs Tools

MCP resources:

- Expose content catalogs (e.g., available document summaries, database schemas, issue hierarchies) that the agent can browse without making action-oriented tool calls
- Reduces exploratory tool calls — instead of the agent calling `list_documents` repeatedly to discover what's available, it reads the resource catalog once

TRAP

03 PART III — Quick Reference and Anti-Patterns

16. Anti-Patterns to Recognize on Sight

ANTI-PATTERN	WHAT IT LOOKS LIKE	WHY IT'S WRONG
Silent failure	Return empty result as successful; include conflicts without flagging them	Corrupts downstream analysis; pipeline looks healthy while output is wrong
Crash-the-plane	Any error terminates entire workflow	Disproportionate; coordinator exists for nuanced recovery
Retry permanent failures	Exponential backoff on corrupted files	Corruption won't self-heal; wastes time and resources
Downstream compensation	Adding summarization agents, vector DBs, or deduplication for upstream problems	Adds complexity; fix at source instead (shift left)
Behavioral over structural	Prompt says "only use for documents" instead of scoping the tool	LLMs don't consistently follow constraints; structural guardrails provide guarantees
Over-engineering	Dedicated error-handling agents, shared state systems, pre-routing classifiers	Each component is a new failure mode; simplest correct solution wins
NL loop termination	Parsing "I'm done" text instead of checking `stop_reason`	Fragile, non-deterministic; `stop_reason` is the authoritative signal
Too many tools	Agent has 18 tools instead of 4–5	Degrades selection reliability; increases misrouting
Generic error status	"Operation failed" without `errorCategory` or `isRetryable`	Coordinator can't make informed recovery decisions
Raw confidence trust	Using model's self-reported confidence without calibration	LLM confidence is poorly calibrated; calibrate against labeled validation data

17. Core Terminology

TERM	DEFINITION
`stop_reason`	API response field indicating why Claude stopped generating. `"tool_use"` = wants to call a tool (loop continues). `"end_turn"` = finished (loop terminates).
Task tool	The Agent SDK mechanism for spawning subagents. Coordinator's `allowedTools` must include `"Task"` to delegate.
`allowedTools`	Configuration restricting which tools an agent can use. Enforces least privilege at the agent level.
`AgentDefinition`	SDK configuration for a subagent: description, system prompt, and tool restrictions.
`PostToolUse` **hook**	Programmatic interceptor that transforms tool results before the model processes them. Used for data normalization.
`fork_session`	Creates independent conversation branches from a shared baseline to explore divergent approaches.
`isError` / `isRetryable`	MCP error response fields. `isError` flags failure; `isRetryable` indicates if retry may succeed.
`errorCategory`	MCP field classifying errors as transient, validation, permission, or business.
`tool_choice`	API parameter: `"auto"` (may skip tools), `"any"` (must call a tool), or forced (must call specific tool).
`.mcp.json`	Project-level MCP server configuration. Shared via version control. Supports `${ENV_VAR}` expansion.
MCP resources	Content catalogs exposed by MCP servers. Agents browse them to discover available data without exploratory tool calls.
Hub-and-spoke	Topology where all agent communication routes through a central coordinator for observability and governance.
Claim-source mapping	

18. 15 Guiding Principles

When uncertain on an exam question, run it through these. The correct answer will align with most.

1. **Code guarantees; prompts suggest.** Enforce via hooks, prerequisites, and tool capabilities — not instructions.
2. `stop_reason` **is the loop signal.** Continue on `"tool_use"`, terminate on `"end_turn"`. Nothing else.
3. **Fix problems at the source.** Redesign upstream outputs rather than adding downstream compensation.
4. **Errors are actionable data.** Include failure type, what was attempted, partial results, and alternatives.
5. **Handle errors at the right level.** Transient locally; permanent escalated with context.
6. **Classify failures semantically.** Access failure ≠ valid empty result ≠ data conflict ≠ partial success.
7. **Context must be explicit.** Subagents don't inherit history. Pass findings directly in prompts.
8. **The coordinator plans; agents execute.** Incomplete output with successful agents = bad decomposition.
9. **Produce value with transparency.** Work with what's available, annotate what's missing.
10. **Least privilege for tools.** 4–5 scoped tools per agent. Too many degrades selection.
11. **Simplest correct solution wins.** Avoid added agents, shared state, classifiers when simpler fixes work.
12. **The hub exists for observability.** Coordinator's value is visibility and governance, not performance.
13. **Structured error responses always.** `isError` + `errorCategory` + `isRetryable` + description.
14. **Calibrate before trusting confidence.** Raw LLM confidence is unreliable; validate against labeled data.
15. **Preserve provenance through every handoff.** Claim-source mappings, dates, and excerpts must survive synthesis.

End of Study Guide

You've written a function. Now we're going to make it fail on purpose, and then fix it. This is how you learn to debug.